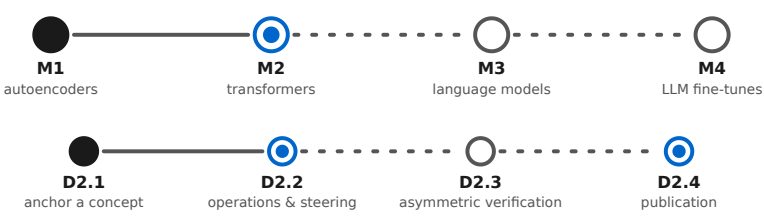
Sparse Concept Anchoring in transformers

Sparse Concept Anchoring (SCA) is a training-time method for concept control. A light geometric regularizer, driven by a small number of noisy labels, guides a chosen concept toward a known location in representation space. The geometry is shaped during training, so there is no need to reverse-engineer it afterwards: the concept lives where you put it, and the side effects of suppressing or ablating it can be bounded before running the intervention.

The first milestone (M1) established the method in autoencoders: paper, blog post, code. This site holds the experiment reports for the second milestone (M2), which asks: does SCA transfer to transformers? We train a small transformer to do vector math and anchor concepts in its residual stream. The vectors are colors, in a synthetic color-mixing task (`red + blue = purple`) where ground truth is unambiguous, so a negative result stays interpretable. The plan and deliverables are in the project README.



- Experiments
- Iteration 0 (prep)
- D2.1: anchoring in a transformer
- D2.2: anchoring an operation
  - Removing *red*
  - A richer grammar for *red*
  - Anchoring an op word, and the pivot
  - Teaching the in-context task
  - Anchoring and removing the inferred op
  - What the edit does, and how we measure it
  - A recipe that keeps the anchor on every run
  - The recipe of record, scored for the post

Experiments

Each report is a literate script.[1] Each entry has searchable tags and a strip of the report's figures, in reading order.

1 A literate script is plain Python with Markdown prose between the cells....

Iteration 0 (prep)

These experiments were preparation for the main work: exercising the infrastructure, testing the normalized transformer architecture, and tying off some loose ends from M1.

▼ **Reports**

nGPT scaling

Width × depth sweep of our simplified nGPT to check that it stays well-behaved as it grows. Converged loss improves with width and is flat across depth, and no condition spikes or stalls. The architecture seems safe to build the color-mixing experiments on.

`ngpt` `scaling` `architecture` `infra`

M1 2.9.1. Deleting *red*, now in JAX

We ported the main M1 result from PyTorch to JAX: Anchor *red* to one latent axis of a small autoencoder, then zero the axis and watch red disappear. The result reproduces, seed sensitivity and all, and our new infrastructure holds up.

`anchoring` `ablation` `jax-port` `seed-sensitivity`

M1 2.9.2. Fallback control

We compared two remedies for ablation seed-sensitivity: **1.** optimal ablation, which picks a replacement constant after training, and **2.** fallback control, a decoder-only loss term that teaches the model what to output once the concept has been removed. Fallback control worked well. Neither remedy touches the other half of the variance, where anchoring itself fails.

`ablation` `fallback-control` `optimal-ablation` `seed-sensitivity`

M1 2.9.3. When anchoring fails

Every failing run anchors first and then comes apart during the high learning-rate plateau. Halving the peak learning rate removes the failures.

`anchoring-failures` `training-dynamics` `learning-rate` `schedules`

M1 2.9.4. Closed-loop regularizer weights

We replaced the timed regularizer anneal with a feedback controller, so that each weight climbs while its constraint is being violated and settles back once it is met. It kind of worked but not very well, and it adds complexity.

`feedback-controller` `regularizer-weights` `schedules`

D2.1: anchoring in a transformer

D2.1 asked whether SCA works in a transformer at all. We designed a task where a small transformer learns the geometry of color on its own, then anchored *red* to a chosen direction in its residual stream. With a repulsive term, a schedule for it, and a pull that finds the red operand by itself, *red* landed where we put it, graded by how red each color is, at no measurable cost to the task.

▼ **Reports**

2.1.1. Un-anchored color-mixing transformer

A small transformer learns a character-level language of color-mixing equations, solving the forms it saw in training and unseen hex pairs. Color turns out to be linearly decodable from its residual stream, with each seed putting *redness* in a different place. Held-out *named* pairs sit at zero accuracy.

`char-tokens` `hex-colors` `color-names` `linear-probes` `generalization`

2.1.2. Making composition necessary

The previous model never answered a held-out *named* pair, so we changed the grammar to make composition the only route: reverse alias lines, and named equations whose mix falls off the palette. The model picks up both new skills and still won't chain them within one forward pass, leaving held-out named accuracy at zero.

`char-tokens` `hex-colors` `color-names` `composition` `grammar-design`

2.1.3. Named colors only

No hex codes, just one opaque token per color and nothing in the text to say that colors are values at all. The model infers the geometry from co-occurrence alone: its embeddings hold the RGB cube as a linear subspace, it computes mixes in value space just before answering, and its held-out guesses land on or beside the right color.

`word-tokens` `named-colors` `embedding-geometry` `vocab-size`

2.1.4. Spelling the names

Does geometry inference need one token per concept? Not at 216 colors. We test the same equations but with every color spelled as an opaque four-letter name. There's a small drop in held-out accuracy, but the value subspace survives. Reading names takes most of the network's depth, leaving one layer for the mix to live in. The 27-color grid is too coarse.

`char-tokens` `spelled-names` `embedding-geometry` `depth`

2.1.5. Disjoint vocabularies and more named colors

Names and hex codes. Both sublanguages train, and each builds its own linear color geometry. But the two geometries resist being combined, even under pressure from a narrow residual stream. Keeping them apart apparently costs less than merging them would.

`char-tokens` `hex-colors` `long-color-names` `disjoint-vocabularies` `embedding-geometry`

2.1.6. Anchoring *red* in a transformer

Our first anchored transformer, with a single attractive term. The anchor moved the activations to the chosen direction, and cost the task nothing measurable. But it moved the whole color cube, rather than *red* in particular.

`word-tokens` `anchoring` `selectivity`

2.1.7. A repulsive term and a narrower pull

We tested two mechanisms to improve anchor selectivity: **1.** Apply the anchor term only to operand 1 (no other tokens), and **2.** Add a repulsive term to clear the target subspace. Both work, but 1. worked better, and their effects stack somewhat.

`word-tokens` `anchoring` `repulsion` `selectivity`

2.1.8. Repulsion tuning

We tested anti-subspace schedules (trailing timing and strength) to find an operating point that contains the cube-wide drift without reducing selectivity. Holding the repulsion high for longer contains the drift and keeps the margin, which nothing before this did. It's unclear if the response is well-graded.

`word-tokens` `anchoring` `repulsion` `schedules` `grading`

2.1.9. Softmin sequence pooling

Anchoring on the first operand alone worked best so far, but in natural language we won't know which tokens hold the concept. So we let the pull choose its own position, by pooling over the line with a soft minimum. It finds the first operand unaided and the anchor stays healthy, though only the softest pooling stays graded in every run.

`word-tokens` `anchoring` `pooling` `mellowmax` `grading`

2.1.10. A label that doesn't point

In ex-2.1.9 the label still only marked the first operand, so the label itself said where the concept was. Here a line is labelled when either operand is red, and the pull has to find the red one line by line. It does, and the anchor is about as selective as when we told it the position.

`word-tokens` `anchoring` `pooling` `weak-labels` `selectivity`

2.1.11. Ablations and a survey for the D2.1 operating point

A survey. The schedules and weights in the recipe were inherited piece by piece and never tuned together, so we tried removing each one, then searched over what was left. The anchor schedule and half the training could go, but the schedule for the repulsive term could not. The search found a wide region of good settings and proposed a stronger anchor at its edge.

`word-tokens` `anchoring` `ablation-study` `sobol-search` `schedules`

2.1.12. Fitted channel probes over the anchored checkpoints

We fitted linear probes to read each operand's color at every depth and position of the D2.1 models. Anchoring costs no color readability that we can measure. The probes also explain an odd pattern in the D2.1 figures, where positions before the second operand seemed to respond to its color: it comes from which test equations we used, not from the model.

`word-tokens` `linear-probes` `checkpoints` `decodability`

D2.1 figures for the anchoring post

Figures for the LessWrong post on D2.1, drawn from the published results above: the anchoring recipe built up one piece at a time, and where in the model the response sits.

`figures` `grading-clouds` `blog-post`

0112 - practicing an opinionation

Practicing an opinionation

0112 - practicing an opinionation

Practicing an opinionation is the act of practicing an opinionation.

Practicing an opinionation means that you take the time and effort to practice an opinionation. You don't just have an opinionation and then never practice it. You practice it. You work on it. You refine it. You make it better.

Practicing an opinionation is important because it helps you to develop your opinionation. It helps you to think more critically about your opinionation. It helps you to see the strengths and weaknesses of your opinionation. And it helps you to communicate your opinionation more effectively.

So, if you have an opinionation, don't just keep it to yourself. Practice it. Work on it. Refine it. And make it better. Your opinionation will thank you for it.

#practicing #opinionation #opinionations #development #critical-thinking #communication

Practicing myself

Practicing myself is the act of practicing myself.

Practicing myself means that you take the time and effort to practice yourself. You don't just exist and then never practice it. You practice it. You work on it. You refine it. You make it better.

Practicing myself is important because it helps you to develop yourself. It helps you to think more critically about yourself. It helps you to see the strengths and weaknesses of yourself. And it helps you to communicate yourself more effectively.

So, if you have yourself, don't just keep it to yourself. Practice it. Work on it. Refine it. And make it better. You will thank yourself for it.

#practicing #myself

`fixed` what it did on `falloff`, and a model that never learned `difference` lands only a little closer to the target null than the edited model. The last piece before the post is a comparison with directions found after training, planned in the post-hoc comparison page.

▼ **Reports**

An absolute tolerance for the write

The write tolerance of ex-2.2.1 is a ratio (a relative tolerance), so where the clean bound is a fraction of a degree a tiny turn counts as over. A re-analysis of four recipes finds an absolute tolerance in how much the bound varies between runs, about a degree. With both, the sites at symbols and separators drop out, and what stays are the operands at the first slice, on a run or so of the recipe of record and on every run of `pool 1–3, soft τ`.

`reanalysis` `intervention` `measurement`

What the edit does on `τ_ℓ fixed`

A re-analysis of the fixed confirmation's runs. The fixed confirmation scored `τ_ℓ fixed` on the full edit alone, and edit-effects scored the earlier recipe; this report gives `τ_ℓ fixed` the scoring of edit-effects, and adds removal by dose and the confusion matrices. Every measure comes out about as on `falloff`. The clean bound at slice 3 is wider, set by a small tail rich in near misses for `difference`, which fits the account of block 1 in block1-edit. For where the edited answers land, it draws on the model trained without the op rather than repeating it.

`reanalysis` `in-context` `intervention` `grading`

A model trained without `difference`

The control trained with no loss on `difference` contexts, so that it never learns the op, against the edited model of the recipe of record. It comes a little closer to the target null than the edited model, but still falls well short, and misses in the same way: the mass goes to the other ops, but it is shared among them differently from the target null on each context, and differently again at each seed.

`in-context` `intervention` `grading` `seed-sensitivity`

What the edit misses in the low runs

A few runs trained with one label draw held for the run end with the edit removing much less of `difference`, and a twin trained again from the same labels, seed, and batches matches each in loss and alignment and still removes the op well. A re-analysis of 48 stored checkpoints: every run also writes, off the anchored axis, whether each example fits the op, and in a low run the reader blocks take part of that judgement from copies the edit never touches, at the answers or at the other example positions. The low runs peak lower and give back more while the anti weight anneals.

`reanalysis` `in-context` `intervention` `seed-sensitivity`

---

1. A literate script is plain Python with Markdown prose between the cells. It reads durable results produced by a separately-run experiment. Reports are published automatically, with their figures served from a Hugging Face dataset; the infrastructure is mi-ni. ↩